

Week 1, Lecture 2: Algorithms in Statistics

Instructor notes – Part 1: prep reading, Part 2: podium cheat sheet

This single file has two parts, both instructor-only (not a student handout): **Part 1** is read beforehand – full worked examples, rendered figures, and the reasoning behind the flow. **Part 2** (starts after the page break, large print) is the podium copy, glanced at while actually teaching – terse bullets, sketch prompts, small reference figures. Part 2 re-plots objects already computed in Part 1 rather than re-simulating, so the two parts can never disagree with each other.

Source material for this lecture is almost entirely mined from the old `LECTURE1.Rmd` (“Computational algorithms”) – previously delivered by live-coding through the knitted HTML, which I’m not doing this year. The worked examples below are adapted from that file rather than invented fresh, since they were already tested in front of students and they’re genuinely fun. WOD2 (permutation pseudocode) was itself lifted from this same old lecture’s lizard example, so today’s lecture and this morning’s WOD are two halves of the same story.

Where this sits in the week

- Monday: syllabus only, no content.
- Tuesday (Lab 1): diagnosed the tadpole predation data, picked the Ricker function as a candidate, hand-computed SSR for it vs. a null model. Students already have “try parameter values, compute a fit score, compare” in their hands.
- Wednesday AM (WOD2): hand-wrote pseudocode (and attempted R) for a permutation test on lizard body mass data, alone/in pairs, no discussion of the answer yet.
- Wednesday lecture (this one): name what they’ve been doing as an *algorithm*, contrast it with analytical solutions, work through two full examples (one of which reveals the WOD2 answer), and preview where this goes for the rest of the semester.

Scientific workflow big-picture: moved to Week 2 Lecture 1

Didn’t get to this live in the actual Wednesday session – the Popperian modeling-loop framing (THEORY/QUESTION → CANDIDATE MODEL(S) → DATA → COMPARE → repeat, Bolker Fig. 1.5) now opens `NOTES_Week2_Lecture1_WorkingWithFunctions.Rmd` instead. Nothing below depends on it – the analytical-vs-computational throughline stands on its own.

Rationale for the flow

The throughline is **analytical vs. computational solutions** (Bolker 1.3.2): an analytical model is solved with algebra/calculus, one formula, done; a computational one is a program you run to see what happens. Rather than teach that distinction abstractly, the salmon example below hits it head-on with two solutions to the *same* problem side by side – a one-line canned test and a from-scratch simulation – so students see the tradeoff concretely before we generalize it.

The lizard example immediately after is not a new concept, it's a *second instance of the same skeleton* (loop -> recompute a statistic -> store -> summarize), applied to a case where we can't even write down a null distribution formula. Revealing the WOD2 answer here, rather than handing it back marked up, makes the lecture feel like a payoff rather than a lecture.

The bootstrap example (trees data) is a stretch goal, not core – cut it first if time is short. It earns its place only because it's a *third* flavor of the same loop (resample rows, not values or labels), which helps students see “algorithm” as a general strategy rather than a trick specific to hypothesis testing.

Case study 1: does a new salmon diet lower body mass?

The setup

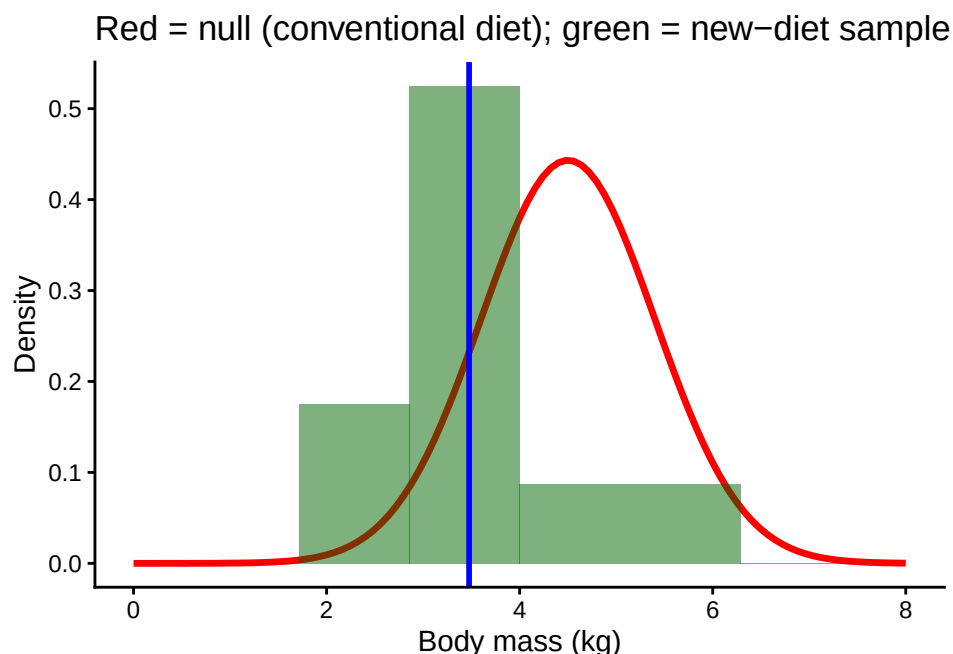
Farm-raised Atlantic salmon on a conventional diet have body mass, after one year, that's well-described by $Y \sim Normal(\mu = 4.5\text{kg}, \sigma = 0.9\text{kg})$. Ten fish were raised instead on a new plant-based diet; we want to know if their mass is lower. This is a good opening example precisely because it's silly and concrete (fish diets) but the statistics are the most familiar thing in the room – a z-test – so all the cognitive load can go toward the *analytical vs. algorithmic* distinction instead of new statistical machinery.

```
pop_mean <- 4.5
pop_sd <- 0.9
my_sample <- c(3.14, 3.27, 2.56, 3.77, 3.34, 4.32, 3.84, 2.19, 5.24, 3.09)
N <- length(my_sample)
sample_mean <- mean(my_sample)
sample_mean
```

```
## [1] 3.476
```

```
ggplot() +
  stat_function(fun = dnorm, args = list(mean = pop_mean, sd = pop_sd),
               color = "red", linewidth = 1.2) +
  geom_histogram(aes(x = my_sample, y = after_stat(density)),
                 bins = 8, fill = "darkgreen", alpha = 0.5) +
  geom_vline(xintercept = sample_mean, col = "blue", linewidth = 1) +
  xlim(0, 8) +
  labs(x = "Body mass (kg)", y = "Density",
```

```
title = "Red = null (conventional diet); green = new-diet sample" +  
theme_classic()
```



Sticking point to flag: this is a one-tailed test (we specifically hypothesized *lower* mass) – worth a quick check-in, since WOD2’s lizard test was two-tailed and it’s easy for students to blur the two together.

Route A: analytical (canned)

The population SD is known, so this is a textbook z-test. Standard error of the sampling distribution of the mean:

$$SE = \sigma / \sqrt{n}$$

```
pop_se <- pop_sd / sqrt(N)  
z_stat <- (sample_mean - pop_mean) / pop_se  
p_value_analytical <- pnorm(sample_mean, pop_mean, pop_se)  
z_stat
```

```
## [1] -3.597969
```

```
p_value_analytical
```

```
## [1] 0.0001603558
```

One formula, exact, instant. This is the *analytical* solution – and it’s the right tool here, because the assumptions (known population SD, normality) are stated as given. **The point is not “simulation is better”** – it’s that we had this option only because the problem was simple enough to solve in closed form.

Route B: brute-force (algorithmic)

Now suppose we'd never heard of a z-test, or forgot the standard-error formula, but understood the story: *if the new diet did nothing, what would sample means look like if we kept re-drawing samples of 10 fish from the conventional population?* We can answer that by literally doing it, many times, in a loop.

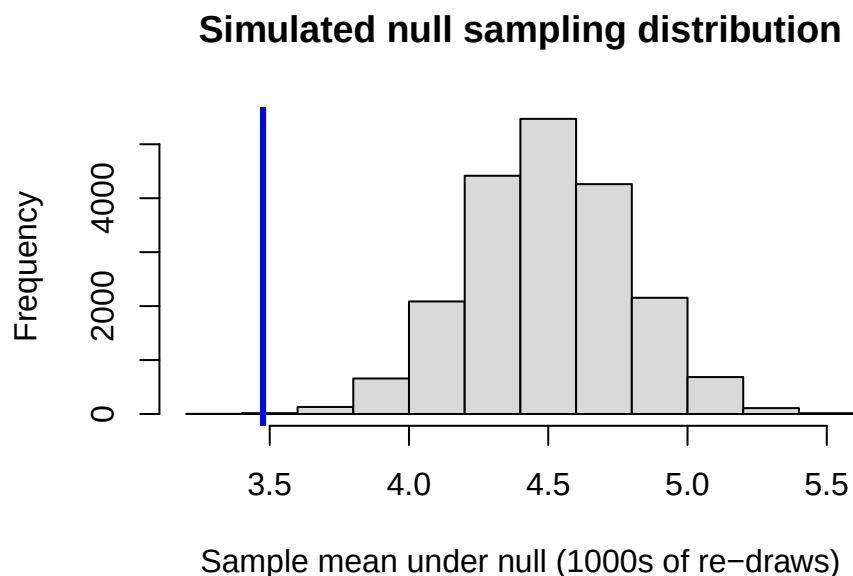
```
set.seed(1)
lots <- 100000
null_population <- rnorm(lots, mean = pop_mean, sd = pop_sd)

reps <- 20000
null_statistics <- numeric(reps)
for (i in seq_len(reps)) {
  null_sample <- sample(null_population, size = N)
  null_statistics[i] <- mean(null_sample)
}

p_value_bruteforce <- sum(null_statistics <= sample_mean) / reps
p_value_bruteforce

## [1] 0.00025

hist(null_statistics, main = "Simulated null sampling distribution",
      xlab = "Sample mean under null (1000s of re-draws)", col = "grey85")
abline(v = sample_mean, col = "blue", lwd = 3)
```



Board pseudocode (this is the one to write up, plain English, same register as WOD2):

1. Simulate a giant “population” under the null hypothesis.
2. Repeat many times: draw a sample of size N , compute its mean, store it.
3. Compare the observed sample mean to this simulated distribution.
4. p -value = fraction of simulated means at least as extreme as observed.

Compare `p_value_analytical` and `p_value_bruteforce` live – they should agree to within simulation noise. That agreement *is* the punchline: the algorithm didn’t need the SE formula at all, it needed only an understanding of the sampling story. That’s the trade: analytical solutions are exact and instant but formula-bound; algorithms are noisier and slower but work even when no formula exists (and we’re about to see a case where none does).

Case study 2: does treatment affect lizard body mass? (WOD2 reveal)

The setup (identical to WOD2)

```
lizard_df <- data.frame(
  trtA      = c(175, 168, 168, 190, 156, 181, 182, 175, 174, 179),
  Control   = c(185, 169, 173, 173, 188, 186, 175, 174, 179, 180)
)
reshape_df <- lizard_df |> pivot_longer(everything(), names_to = "Treatment", values_to = "Mass")
reshape_df$Treatment <- factor(reshape_df$Treatment, levels = c("Control", "trtA"))

observed_dif <- diff(with(reshape_df, tapply(Mass, Treatment, mean)))
observed_dif

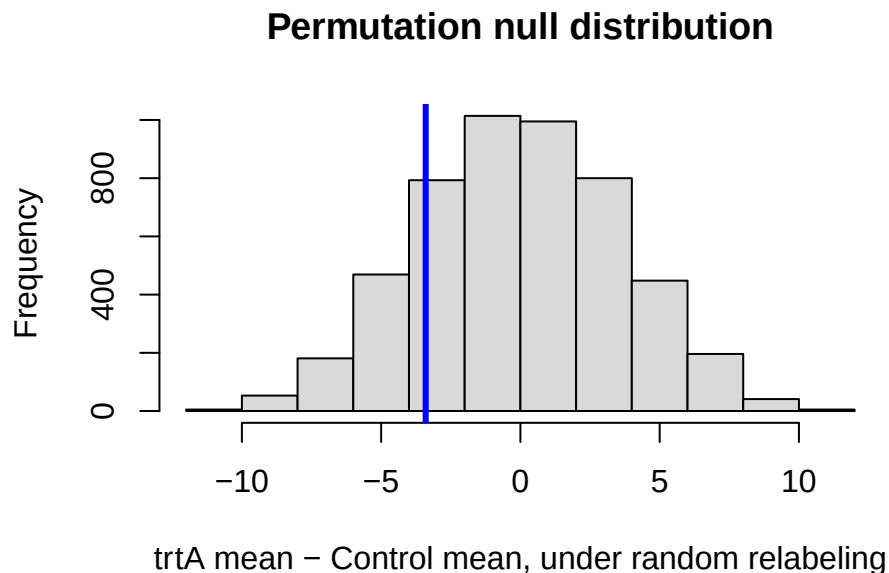
## trtA
## -3.4
```

This time there’s no known population distribution to simulate from – we don’t want to *assume* normality at all. Instead of simulating new data, we **reshuffle the labels** on the data we already have. If treatment truly doesn’t matter, the labels are meaningless, and reshuffling them shouldn’t systematically change the group-mean difference.

```
set.seed(1)
reps <- 5000
null_difs <- numeric(reps)
for (i in seq_len(reps)) {
  shuffled <- sample(reshape_df$Treatment)
  null_difs[i] <- mean(reshape_df$Mass[shuffled == "trtA"]) -
    mean(reshape_df$Mass[shuffled == "Control"])
}
p_value_perm <- sum(abs(null_difs) >= abs(observed_dif)) / reps
p_value_perm
```

```
## [1] 0.3828
```

```
hist(null_difs, main = "Permutation null distribution",  
      xlab = "trtA mean - Control mean, under random relabeling", col = "grey85")  
abline(v = observed_dif, col = "blue", lwd = 3)
```



Reveal moment: put this next to their WOD2 fill-in-the-blank function and let them check their own pseudocode/R attempt against it. The only structural difference from the salmon algorithm: step 2 becomes “reshuffle labels” instead of “simulate a new sample.” Everything else – loop, recompute, store, summarize into a p-value – is identical. That repetition is the actual lesson, more than either example alone.

```
permutation_test <- function(data, group, value, reps){  
  observed_dif <- diff(tapply(data[[value]], data[[group]], mean))  
  null_difs <- numeric(reps)  
  for(i in seq_len(reps)){  
    shuffled_labels <- sample(data[[group]])  
    null_difs[i] <- diff(tapply(data[[value]], shuffled_labels, mean))  
  }  
  p_value <- sum(abs(null_difs) >= abs(observed_dif)) / reps  
  return(p_value)  
}
```

Bonus (cut first if short on time): bootstrapping a confidence interval

A third flavor of the same idea, worth a quick mention even if not fully worked: does tree girth or height better predict volume? R^2 is easy to compute for each, but no base R function hands you a confidence interval on R^2 .

```
r_squared <- function(data, responsevar = "Volume") {
  preds <- setdiff(names(data), responsevar)
  sapply(preds, function(p) {
    m <- lm(as.formula(paste0(responsevar, "~", p)), data)
    summary(m)$r.squared
  })
}
r_squared(trees, "Volume")
```

```
##      Girth      Height
## 0.9353199 0.3579026
```

```
set.seed(1)
boot_sample <- function(data, fun, nboot, y) {
  t(replicate(nboot, fun(data[sample.int(nrow(data), replace = TRUE), ], responsevar = y)
})
boot <- boot_sample(trees, r_squared, 1000, "Volume")
apply(boot, 2, function(x) quantile(x, c(0.025, 0.5, 0.975)))
```

```
##           Girth      Height
## 2.5%  0.8968796 0.1393925
## 50%   0.9379984 0.3569473
## 97.5% 0.9619022 0.5942177
```

Same skeleton again: resample (this time *rows*, with replacement), recompute a statistic, store it, repeat, summarize – now the summary is a quantile-based interval instead of a p-value. If this comes up, that’s the only point worth making; don’t get pulled into explaining bootstrap theory today.

Take-home message (verbatim spirit of the old lecture’s ending)

- Flexibility: if you understand the story, you can build an algorithm for it, even with no formula in sight.
- You can relax assumptions a canned test forces on you (and writing the simulation forces you to state your assumptions explicitly).
- Runs analyses that don’t otherwise exist as a “canned” option.
- It’s fun.

- With great power comes great responsibility – flag this lightly, will return to it around model selection / overfitting later in the semester, no need to develop it today.

Roadmap tie-in

Same loop skeleton (guess/resample, recompute, store, summarize) reappears all semester:

- Grid search / optimization for maximum likelihood (Weeks 6, 8)
- MCMC for Bayesian posteriors (Week 9)
- Parametric bootstraps for goodness-of-fit (later, model-checking weeks)

Worth saying explicitly: today’s “brute force” loop is not a beginner trick we’ll graduate away from – it’s the same machine we’ll keep sharpening all semester.

Discussion seed (if time remains)

Ask: *where else in your own research might “I understand the story but there’s no formula” show up?* Let a couple of answers surface without over-scaffolding – the goal is just to plant the idea that this is a general strategy, not a one-off trick for two textbook examples.

PART 2 – PODIUM CHEAT SHEET

Everything above this line is prep reading. From here on is the terse copy for the podium – large print, sketch prompts, small reference figures reused from Part 1’s computations above.

0. Open (~2 min)

- WOD2 $\rightarrow$ you hand-wrote an algorithm
- Lab 1 (Ricker curve, SSR) $\rightarrow$ same idea
- Today: name it. Two case studies. Where it leads.
- (Big-picture Popperian loop moved to open Week 2 Lecture 1 instead)
- **Key pair for today: analytical $\leftrightarrow$ computational**
- Not a math class – we keep going when the algebra runs out

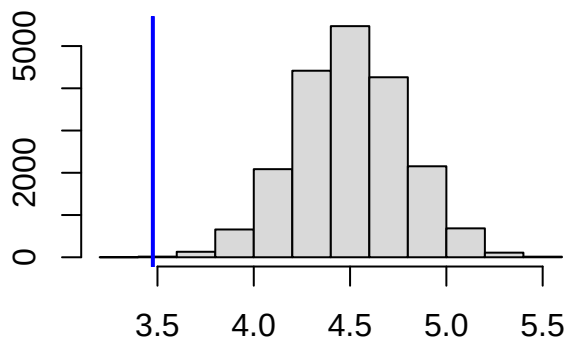
1. Case study 1: salmon diet

- Plant-based diet, $n = 10$, null: $N(\mu = 4.5, \sigma = 0.9)$ kg
- One-tailed (predicted *lower* mass)

Route A – analytical

- $SE = \sigma/\sqrt{n}$
- $z = -3.6, \quad p = 1.6 \times 10^{-4}$
- one formula, exact, instant

Route B – brute force



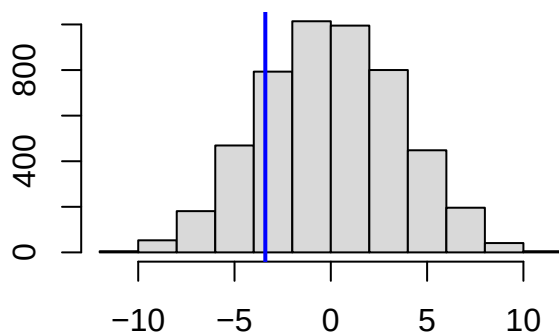
- simulate giant null population
- loop 1000s×: sample $\rightarrow$ mean $\rightarrow$ store
- $p = \text{fraction} \leq \text{observed} \approx 2.5 \times 10^{-4}$

Board pseudocode:

1. simulate population under H_0
 2. repeat: draw sample, compute mean, store
 3. compare observed to stored distribution
 4. $p = \text{fraction as/more extreme}$
- **Punchline:** A and B agree – B needed no formula, just the story

2. Case study 2: lizard reveal (WOD2!)

- Callback: *“you wrote this by hand this morning”*
- Same trtA/Control data – no distribution assumed this time
- Shuffle **labels** instead of simulating new data



- observed diff = -3.4 , $p \approx 0.38$ – **not** significant (nice contrast w/ salmon!)
- same skeleton as case 1, only step 2 differs (reshuffle vs. simulate)
- reveal real R function next to their WOD2 blanks

3. Bonus, if time (~3 min)

- Trees: girth or height $\rightarrow$ predict volume?
- R^2 has no formula for a CI $\rightarrow$ bootstrap it
- resample **rows** w/ replacement, refit, recompute, repeat, quantiles
- 3rd flavor, same skeleton

4. Take-home (~3 min)

- flexibility – no formula, no problem
- relax assumptions; forces you to state them explicitly
- runs analyses “canned” tools can’t
- it’s fun
- great power $\rightarrow$ great responsibility (data dredging – more later)

5. Roadmap (~2-3 min)

- functions (wk2) $\rightarrow$ probability (wk3) $\rightarrow$ simulation (wk4) $\rightarrow$ OLS/likelihood (wk5-6) $\rightarrow$ optimization (wk8) $\rightarrow$ MCMC (wk9) $\rightarrow$ hierarchical/GAM/spatial/dynamic (wk10+)
- same skeleton returns as: optimization, MCMC, parametric bootstrap

If time: discussion seed

- *Where in YOUR research: “understand the story, no formula”?*